Distributed Machine Learning with Spark Apply

Dom Richards

2025-10-06

```
library(sparklyr)
library(dplyr)
library(nycflights13)
```

```
sc <- spark_connect(master = "local", version = "3.4.0")
```

```
flights_tbl <- sparklyr::copy_to(sc, nycflights13::flights, "flights", overwrite = TRUE)
airlines_tbl <- sparklyr::copy_to(sc, nycflights13::airlines, "airlines", overwrite = TRUE)
```

Distributed Machine Learning with Spark Apply

In this section we'll use `spark_apply()` to fit arbitrary R models (such as linear modelling `lm()`) to Spark DataFrames.

This is an incredibly powerful feature of sparklyr, and it's unmatched by other languages such as Python or Java.

As we've already discussed, R is the lingua franca of statistics and data science, with a vast ecosystem of packages for specialized analyses, that simply aren't available in other languages.

R models aren't made for big datasets, so they're best applied to smaller datasets that have been created with `group_by`, e.g. grouping over carriers in our flights dataset.

We'll see an example of this below.

Preparing a minimal dataset

The following code prepares a minimal dataset for distributed machine learning by carrier:

```
# Keep only needed columns and drop NAs
flights_small <- flights_tbl %>%
  select(carrier, dep_delay, arr_delay) %>%
  filter(!is.na(dep_delay), !is.na(arr_delay))

# Register as a temp view and cache/persist in Spark
flights_small <- flights_small %>%
  sdf_register("flights_small") %>%
  sdf_persist(storage.level = "MEMORY_ONLY")
```

The above code performs the following steps:

- `select(carrier, dep_delay, arr_delay)` keeps only the three columns needed for analysis.
- `filter(!is.na(dep_delay), !is.na(arr_delay))` removes rows with missing values in delay columns.
- `sdf_register("flights_small")` creates a temporary Spark SQL table named `"flights_small"` for querying.
- `sdf_persist(storage.level = "MEMORY_ONLY")` caches the cleaned data in memory to speed up subsequent operations and avoid recomputation.

The `sdf_` Family of Functions

The `sdf_` prefix stands for "Spark DataFrame" and indicates functions that operate at the Spark level (not dplyr).

These functions provide low-level control over Spark DataFrame operations that aren't available through dplyr verbs.

They generally access the Scala Spark DataFrame API directly, as opposed to the dplyr interface which uses Spark SQL.

Key `sdf_` functions include:

- `sdf_register(name)`: Creates a named temporary SQL table in Spark's catalog, allowing you to query it with `spark_sql()`.
- `sdf_persist(storage.level)`: Caches the DataFrame in memory/disk to avoid recomputation.
- `sdf_repartition(partitions, partition_by)`: Reorganizes data across cluster partitions for parallel processing.
- `sdf_sample(fraction, replacement, seed)`: Performs random sampling at the Spark level.
- `sdf_schema()`: Returns the DataFrame schema (column names and types).
- `sdf_num_partitions()`: Returns the number of partitions in the DataFrame.

Random sampling and partitioning

The following code performs random sampling and partitioning of the flights data for distributed machine learning:

```
sample_fraction <- 0.01
sample_seed <- 42

flights_frac <- flights_tbl %>%
  sdf_sample(fraction = sample_fraction, replacement = FALSE, seed = sample_seed) %>%
  sdf_repartition(partition_by = "carrier") %>%
  sdf_register("flights_frac") %>%
  sdf_persist(storage.level = "MEMORY_ONLY")
```

The above code performs the following steps:

- `sample_fraction <- 0.01` sets the sample size to 1% of the data for faster computation.
- `sample_seed <- 42` ensures reproducible random sampling.
- `sdf_sample()` performs random sampling on the Spark DataFrame without replacement.
- `sdf_repartition(partition_by = "carrier")` reorganizes data so each carrier's flights are in their own partition, enabling efficient group-wise operations.
- `sdf_register("flights_frac")` creates a temporary SQL table for later queries.
- `sdf_persist(storage.level = "MEMORY_ONLY")` caches the sampled data in memory to avoid recomputation.

Why `sdf_repartition()` AND `group_by()`?

You might wonder why we use both `sdf_repartition(partition_by = "carrier")` here and `group_by(carrier)` later.

These serve different but complementary purposes:

- `sdf_repartition(partition_by = "carrier")` is physical data organization: It moves data across the cluster so all rows with the same carrier are stored on the same worker node partition.
- `group_by(carrier)` is logical grouping: It tells `spark_apply()` which column defines the groups for function application.

Why use both? Without repartitioning, `group_by()` + `spark_apply()` would need to shuffle data across the network to group carriers together. The repartitioning pre-positions the data optimally, making the distributed operation much faster.

Sanity check

Before proceeding with analysis, we perform a quick sanity check to verify the sample size:

```
flights_frac %>%
  count() %>%
  collect()
```

```
## # A tibble: 1 x 1
##       n
##   <dbl>
## 1  3337
```

Define output schema

Next we need to define the schema of the output DataFrame that will be returned by `spark_apply()`:

```
column_names <- list(
  carrier = "character",
  n_flights = "integer",
  intercept = "double",
  intercept_pval = "double",
  slope = "double",
  slope_pval = "double",
  r_squared = "double"
)
```

Distributed Linear Regression via `spark_apply(lm)`

```
carrier_lm_results <- flights_frac %>%
  group_by(carrier) %>%
  spark_apply(function(df) {
    model <- lm(arr_delay ~ dep_delay, data = df)
    model_summary <- summary(model)
```

```
    coefs <- model_summary$coefficients
    data.frame(
      carrier = df$carrier[1],
      n_flights = nrow(df),
      intercept = round(coef(model)[1], 2),
      intercept_pval = round(coefs[1, 4], 4),
      slope = round(coef(model)[2], 2),
      slope_pval = round(coefs[2, 4], 4),
      r_squared = round(model_summary$r.squared, 3)
    )
  }, columns = column_names) %>%
  arrange(carrier) %>%
  left_join(airlines_tbl, by = "carrier") %>%
  select(name, everything()) %>%
  collect()
```

```
## Warning: ORDER BY is ignored in subqueries without LIMIT
## i Do you need to move arrange() later in the pipeline or use window_order() instead?
## ORDER BY is ignored in subqueries without LIMIT
## i Do you need to move arrange() later in the pipeline or use window_order() instead?
```

Model Equation and Parameter Interpretation

The linear model fitted for each carrier follows the equation:

$$\text{arr_delay} = \beta_0 + \beta_1 \times \text{dep_delay} + \epsilon$$

Where:

- Intercept (β_0): The expected arrival delay when departure delay is zero.
- Slope (β_1): The change in arrival delay for each additional minute of departure delay.
- R-squared: The proportion of variance in arrival delay explained by departure delay.

The OLS estimates are computed using the normal equations, which you derived in the week 2 tutorial:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

Explanation of the code

This code demonstrates distributed linear regression using `spark_apply()` to fit separate models for each carrier:

- `group_by(carrier)` partitions the data by airline carrier, preparing for per-group analysis.
- `spark_apply(function(df) {...})` applies an R function to each partition in parallel across the Spark cluster.
- `lm(arr_delay ~ dep_delay, data = df)` fits a standard R linear regression model for each carrier's subset of data.
- `summary(model)$coefficients` extracts the coefficient matrix containing estimates, standard errors, t-statistics, and p-values.

- `coefs[1, 4]` and `coefs[2, 4]` extract p-values for intercept and slope from the 4th column of the coefficient matrix.
- `data.frame(...)` packages model results into a structured output.
- `columns = column_names` specifies the schema for the returned Spark DataFrame (defined earlier).
- `left_join(airlines_tbl, by = "carrier")` adds human-readable airline names to the results.

Performance Optimization: Pre-partitioned Data

- Because we used `sdf_repartition(partition_by = "carrier")` earlier, the `group_by(carrier)` operation in our `spark_apply()` will be very efficient.
- Without repartitioning: Spark would need to shuffle data across the network when `group_by()` is called, moving all rows with the same carrier to the same worker.
- With repartitioning: Data is already physically co-located on the correct worker nodes, so `group_by()` executes nearly instantaneously with zero network shuffle.
- Result: Much faster execution.
- This is a common optimization pattern in Spark: pre-arrange your data layout to match your computation pattern.

Results

These are the linear regression results by carrier:

```
carrier_lm_results
```

```
## # A tibble: 11 x 8
##    name    carrier n_flights intercept intercept_pval slope slope_pval r_squared
##    <chr>   <chr>       <int>     <dbl>          <dbl> <dbl>      <dbl>     <dbl>
##  1 Virgin~ VX             69    -16.6          0       1.04     0          0.903
##  2 Delta ~ DL            515     -7.47         0       1.01     0          0.781
##  3 JetBlu~ B6            549     -3.55         0       1.01     0          0.743
##  4 US Air~ US            191      0.49         0.731   1.1      0          0.71
##  5 United~ UA            572    -10.2          0       1.04     0          0.756
##  6 Endeav~ 9E            192     -6.13         0       1        0          0.894
##  7 Southw~ WN            116     -6.31         0.0019  1.01     0          0.872
##  8 Fronti~ F9             10     -1.85         0.818   0.78     0.0603     0.374
##  9 Expres~ EV            541     -4.53         0       1.04     0          0.932
## 10 Envoy ~ MQ            262     -0.3          0.792   1        0          0.796
## 11 Americ~ AA            320     -9.53         0       1.05     0          0.617
```

Interpretation

- Slope ≈ 1.0: Most carriers show slopes close to 1, meaning departure delays translate almost 1:1 into arrival delays.
- Negative Intercepts: Many carriers have negative intercepts (e.g., AA: -8.89, DL: -6.72), suggesting flights can make up some time during flight, which is independent of departure delay.
- High R-squared: Values like 0.80+ indicate departure delay is a strong predictor of arrival delay.
- Carrier Differences:
  - Frontier Airlines shows unusual pattern with positive intercept (5.29) but still high correlation (0.98).
  - Virgin America shows the most negative intercept (-11.28), suggesting good in-flight time recovery.

```
spark_disconnect(sc)
```